

ПРОГРАММНАЯ ДОКУМЕНТАЦИЯ

на подсистему распознавания жестов рук
программного комплекса «Система управления компьютером
с помощью жестов рук»

разработанная на основе

SWITCH-ТЕХНОЛОГИИ

2026

СОДЕРЖАНИЕ

1.Введение	3
2.Частное техническое задание на подсистему распознавания жестов	4
3.Структурная схема программного обеспечения	7
4.Перечень и нумерация событий	9
5.Перечень и нумерация входных переменных	10
6.Перечень и нумерация выходных воздействий	11
7.Пояснения к используемой нотации	12
8.Системозависимая часть	15
9.Исходные тексты системозависимой части	26
10.Протоколы функционирования подсистемы	47
11.Требования к программному обеспечению	51
12.Требования к программной документации	53
13.Технико-экономические показатели	54
14.Стадии и этапы разработки	55
15.Порядок контроля и приёмки	56
16.Установка и эксплуатация	57
Прил. А .Допустимые имена клавиш PyAutoGUI	58
Прил. Б. Индексы landmarks руки MediaPipe	59

1. ВВЕДЕНИЕ

Настоящий документ описывает программный комплекс «Система управления компьютером с помощью жестов рук». Назначение системы — бесконтактное управление рабочей станцией через веб-камеру: пользователь показывает одну или две руки, программа определяет число поднятых пальцев и направление движения ладони, после чего эмулирует сочетания клавиш или медиаклавиши средствами PyAutoGUI.

Документация оформлена по методике SWITCH-технологии (автоматный подход к описанию программного обеспечения). Логика распознавания разбита на три координирующих автомата A0, A1 и A2, реализованных методами класса GestureController в модуле gestures.py.

1.1. Состав поставки

Компонент	Файл	Назначение
Точка входа (консоль)	main.py	Захват с камеры, отображение OpenCV
Точка входа (GUI)	gui_app.py	Меню, камера в окне, редактор конфигурации
Ядро распознавания	gestures.py	Класс GestureController, автоматы A0–A2
Конфигурация	gestures_config.json	Таблица «жест → действие»
Зависимости	requirements.txt	Версии библиотек
Документация	doc/	Настоящий документ и блок-схемы

1.2. Режимы запуска

Консольный режим: `python main.py` — окно «Gesture Control», выход клавишей `q`.

Графический режим: `python gui_app.py` — главное меню, отслеживание и редактор жестов.

2. ЧАСТНОЕ ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПОДСИСТЕМУ РАСПОЗНАВАНИЯ ЖЕСТОВ

2.1. Назначение подсистемы

1. Захват видео с веб-камеры в реальном времени (индекс устройства 0).
2. Обнаружение и отслеживание 21 ключевой точки на каждой кисти (MediaPipe Holistic).
3. Подсчёт поднятых пальцев (0–5) с учётом ориентации ладони.
4. Определение направления движения ладони: UP, DOWN, LEFT, RIGHT.
5. Сопоставление тройки (рука, пальцы, направление) с записью в `gestures_config.json`.
6. Выполнение действия через `pyautogui.hotkey(...)`.
7. Визуальную обратную связь (наложение скелета руки, текст состояния).

2.2. Функциональные требования

2.2.1. Обнаружение рук

Одновременно обрабатываются до двух рук: `right_hand_landmarks` и `left_hand_landmarks`. Пороги детекции Holistic: `min_detection_confidence=0.5`, `min_tracking_confidence=0.5`.

2.2.2. Подсчёт пальцев

Для каждой обнаруженной руки вызывается `count_fingers(hand_landmarks)`. Большой палец сравнивается по оси X с учётом `is_right_hand`; остальные — по оси Y (кончик выше сустава означает поднятый палец).

2.2.3. Определение движения

Активная рука: правая при `fingers > 0`, иначе левая. Позиция ладони — среднее между запястьем (0) и MCP среднего пальца (9). История хранится в deque длиной 20; анализ начинается при накоплении не менее `HISTORY_LENGTH // 2` кадров. Порог движения по умолчанию `movement_threshold=80`.

2.2.4. Сопоставление с действием

Линейный поиск по массиву JSON-объектов. Поля записи: `hand`, `fingers`, `direction`, `action`.

2.3. Требования к производительности

Показатель	Значение
Частота обработки	≥ 15 FPS (типично 25–30 на CPU)
Задержка жест $\rightarrow$ действие	$\leq 0,3$ с при накопленной истории
Разрешение	640×480 и выше

2.4. Предусмотренные жесты

Рука	Пальцы	Направление	Действие
left_hand	3	DOWN/UP	win, d
left_hand	2	RIGHT	alt, shift, tab
left_hand	2	LEFT	alt, tab
right_hand	5	DOWN	volumedown

right_hand	5	UP/RIGHT	volumeup
right_hand	4	RIGHT/LEFT	nexttrack / prevtrack
right_hand	4	UP/DOWN	playpause / pause

3. СТРУКТУРНАЯ СХЕМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

3.1. Общая структурная схема программ, разрабатываемых с использованием SWITCH-технологии

3.1. Общая структурная схема ПО

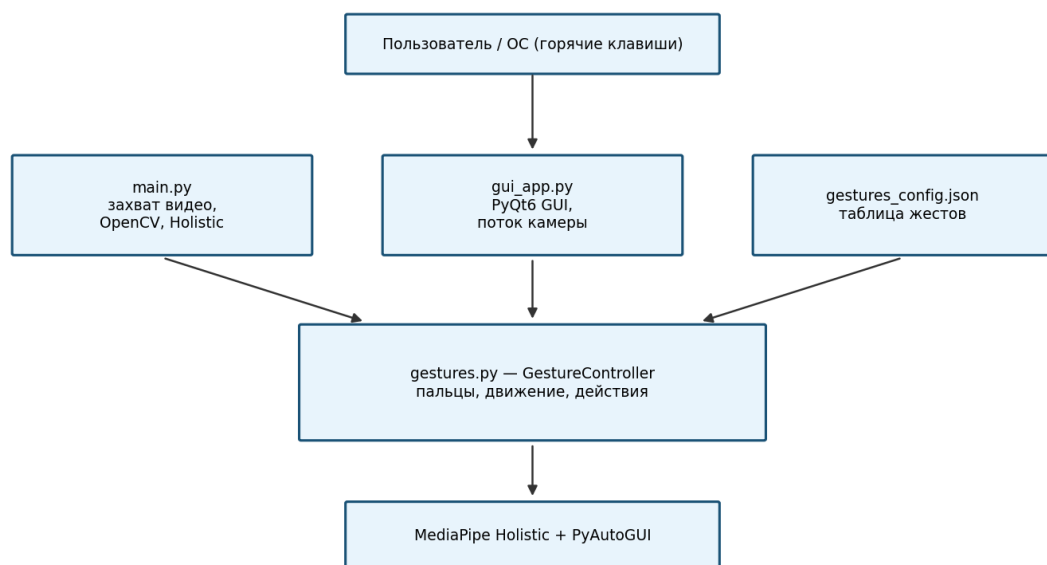


Рисунок 1 — Общая структурная схема программного комплекса

Модуль	Ответственность
main.py	VideoCapture, цикл кадров, Holistic, отрисовка, process_hands
gui_app.py	PyQt6: меню, VideoThread, редактор JSON
gestures.py	GestureController: автоматы A0–A2, загрузка конфига
gestures_config.json	Внешняя таблица соответствий

3.2. Порядок взаимодействия частей подсистемы

3.2. Порядок взаимодействия модулей (один кадр)

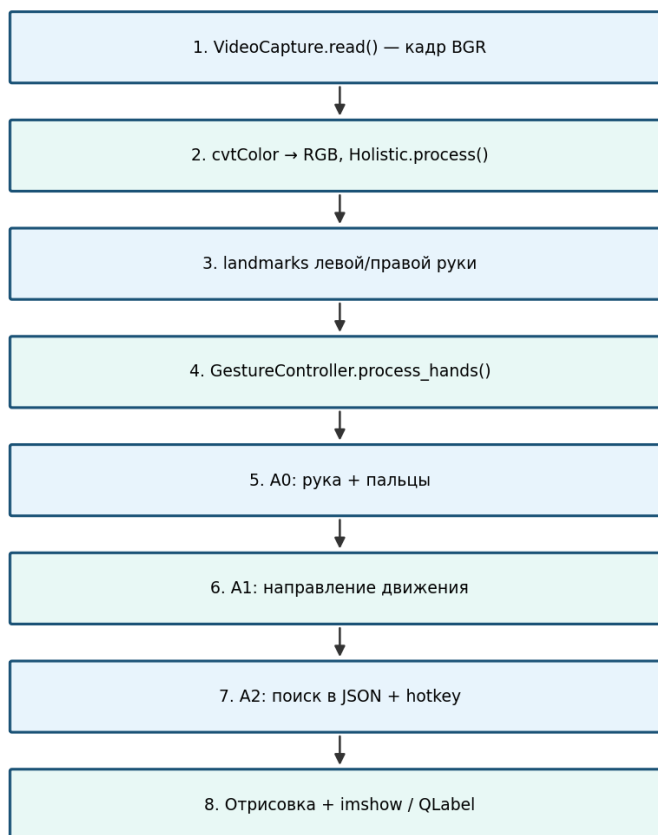


Рисунок 2 — Взаимодействие модулей за один кадр

1. `capture.read()` → кадр BGR.
2. Преобразование в RGB, `holistic_model.process(image)`.
3. Передача `right_hand_landmarks`, `left_hand_landmarks` в `process_hands`.
4. Внутри контроллера: $A0 \rightarrow A1 \rightarrow A2$ (при выполнении условий).
5. Отрисовка `landmarks`, `flip`, текст, `imshow` / сигнал в GUI.

3.3. Блок-схема главного цикла `main.py`

Блок-схема главного цикла (`main.py`)

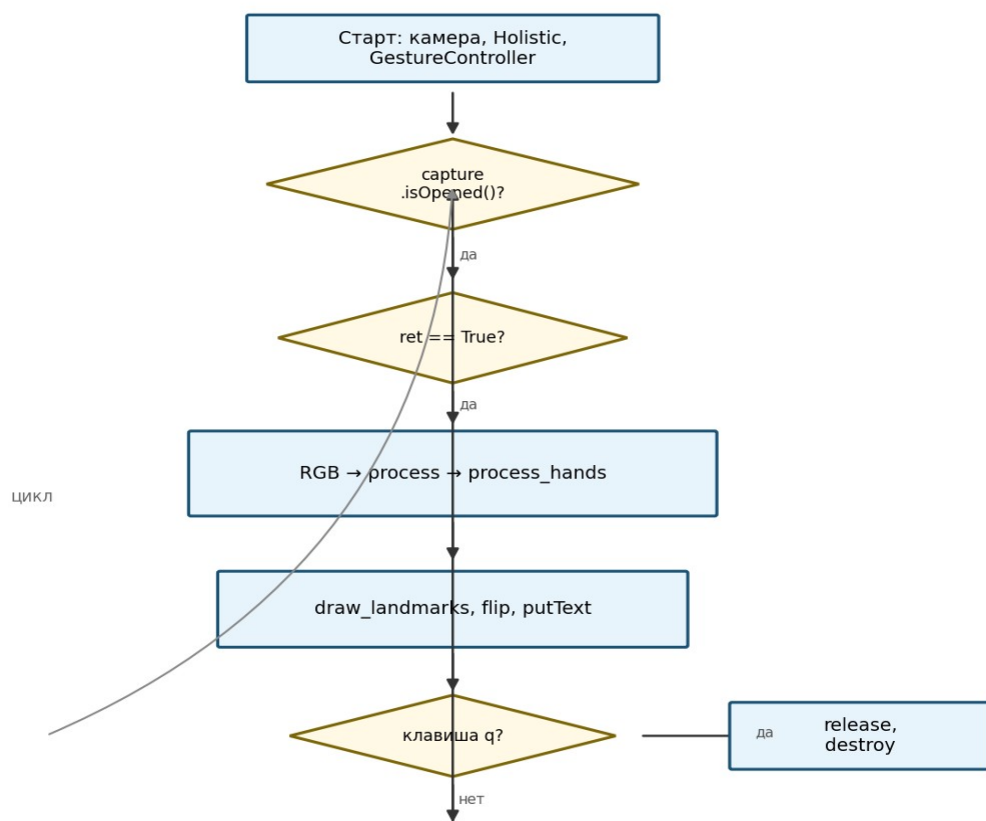


Рисунок 3 — Главный цикл консольного приложения

3.4. Архитектура графического интерфейса

Структура графического интерфейса (gui_app.py)

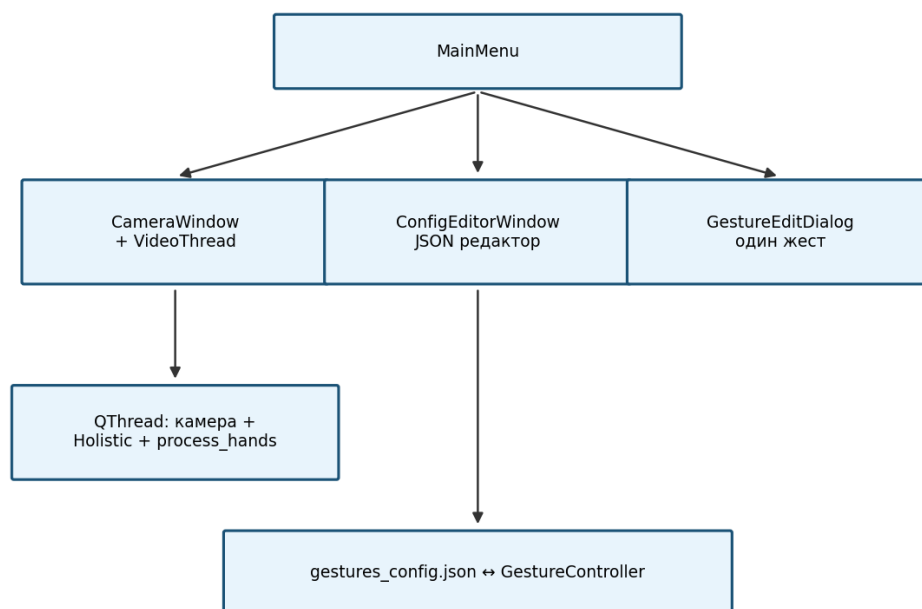


Рисунок 4 — Структура GUI (PyQt6)

4. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ СОБЫТИЙ

Номер	Обозначение	Описание	Источник
m18	NEW_FRAME	Захвачен новый кадр	capture.read()
m63	RIGHT_HAN D	Обнаружена правая рука	right_hand_landmarks
m73	LEFT_HAND	Обнаружена левая рука	left_hand_landmarks
g50	FINGERS_CHANGED	Изменилось число пальцев	current_fingers_count
g211u	MOVE_UP	Движение вверх	detect_movement → UP
g211d	MOVE_DOWN N	Движение вниз	detect_movement → DOWN
g199l	MOVE_LEFT	Движение влево	detect_movement → LEFT
g199r	MOVE_RIGHT T	Движение вправо	detect_movement → RIGHT
g414	HISTORY_READY	Накоплена история $\geq$ HISTORY/2	len(hand_positions)
g268	ACTION_DONE	Выполнено действие	perform_action
m91	KEY_Q	Выход из приложения	waitKey == q

5. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВХОДНЫХ ПЕРЕМЕННЫХ

Номер	Обозначение	Тип	Описание
g92r	right_hand_landmarks	NormalizedLandmarkList	Точки правой руки
g92l	left_hand_landmarks	NormalizedLandmarkList	Точки левой руки
g413	MOVEMENT_THRESHOLD	int	Порог смещения (80)
g415	ACTION_COOLDOWN	float	Задержка между действиями
g416	HISTORY_LENGTH	int	Размер deque (20)
g41	config_data	list[dict]	Загруженный JSON
g17	config_file	str	Путь к конфигурации

6. ПЕРЕЧЕНЬ И НУМЕРАЦИЯ ВЫХОДНЫХ ВОЗДЕЙСТВИЙ

Номер	Обозначение	Описание
m63	show_hand_info	Текст r: N f / l: N f на кадре
m91	show_fingers	Fingers: N
m89	show_direction	Gesture: UP/DOWN/...
g268	execute_action	pyautogui.hotkey(*action)
g92	set_active_hand	current_hand, gesture_active=True
m90	draw_landmarks	Отрисовка HAND_CONNECTIONS
s01	gui_gesture_signal	gesture_info_signal в PyQt

7. ПОЯСНЕНИЯ К ИСПОЛЬЗУЕМОЙ НОТАЦИИ

7.1. Нотация, используемая в графах переходов

Состояния — прямоугольники; условия — ромбы; дуги — стрелки с метками событий (m^* , g^*).

7.2. Схема универсального алгоритма реализации автоматов

Автоматы A0–A2 не выделены в отдельные классы; переходы реализованы последовательными вызовами в process_hands с ранним return при невыполнении условий.

Шаблон Си-функции, реализующей автомат:

```
hand_info = self.detect_hand_and_fingers(...) # A0
if not hand_info: return None
current_position = self.get_hand_position(...)
self.hand_positions.append(current_position)
if len(self.hand_positions) >= self.HISTORY_LENGTH // 2:
    direction = self.detect_movement(...) # A1
```

```
action = self.get_action_from_config(...) # A2  
if action: self.perform_action(action)
```

7.3. Шаблон реализации автомата в коде Python

Каждый автомат соответствует группе методов GestureController; блок-схемы приведены в разделе 8.

8. СИСТЕМОЗАВИСИМАЯ ЧАСТЬ

8.1. Автомат выбора активной руки и подсчёта пальцев (A0)

8.1.1. Словесное описание

S0: рук в кадре нет → gesture_active=False, возврат None.

S1: вызов detect_hand_and_fingers — словарь {right_hand: n, left_hand: m}.

S2: для каждой руки count_fingers.

S3: приоритет правой руки с fingers>0, иначе левая; иначе S0.

S4: сохранение current_hand, current_fingers_count, отображение на кадре.

8.1.2. Схема связей и граф переходов

8.1. Автомат А0 — выбор руки и подсчёт пальцев

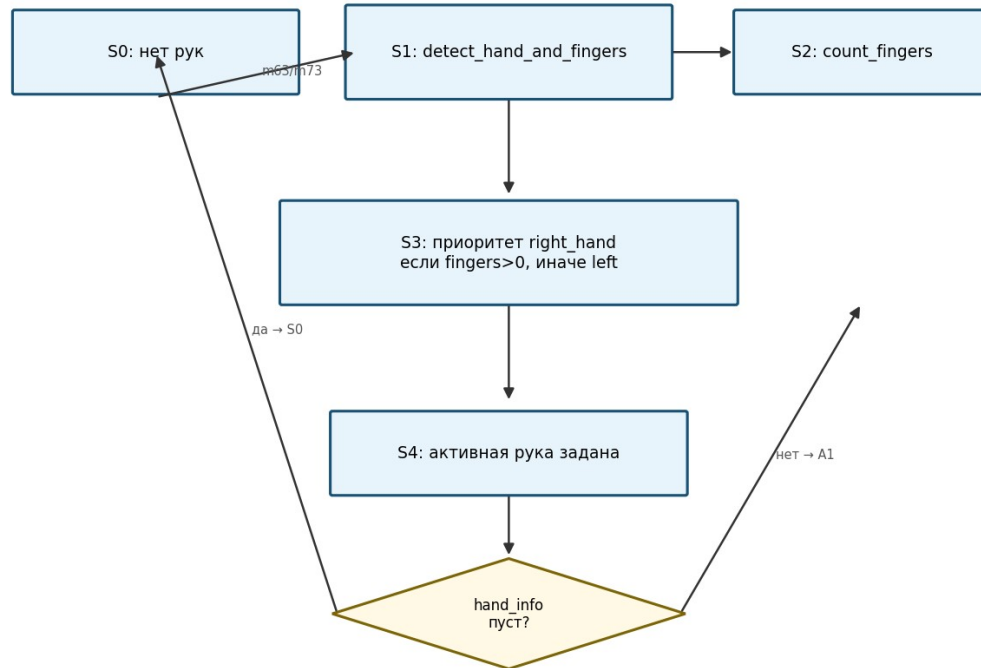


Рисунок 5 — Автомат А0 (выбор руки и подсчёт пальцев)

8.1.3. Текст функции, реализующей автомат

Фрагмент файла `gestures.py` (`count_fingers`, `detect_hand_and_fingers`):

```
def count_fingers(self, hand_landmarks):
    """Подсчет поднятых пальцев на руке"""
    if hand_landmarks is None:
        return 0

    # Индексы ключевых точек для пальцев
    finger_tips = [4, 8, 12, 16, 20] # Кончики пальцев
    finger_pips = [3, 6, 10, 14, 18] # Суставы

    finger_count = 0

    # Определяем активную руку (левую или правую)
    # Для этого используем позицию запястья относительно среднего пальца
    # Если запястье находится левее среднего пальца, то это правая рука, иначе левая
    wrist_x = hand_landmarks.landmark[0].x
    middle_mcp_x = hand_landmarks.landmark[9].x

    is_right_hand = wrist_x < middle_mcp_x
```

```

# Большой палец - сравниваем по оси X в зависимости от руки
if is_right_hand:
    # Для правой руки
    if hand_landmarks.landmark[4].x > hand_landmarks.landmark[3].x:
        finger_count += 1
else:
    # Для левой руки
    if hand_landmarks.landmark[4].x < hand_landmarks.landmark[3].x:
        finger_count += 1

# Остальные пальцы - сравниваем по оси Y (кончик выше сустава = палец поднят)
for tip, pip in zip(finger_tips[1:], finger_pips[1:]):
    if hand_landmarks.landmark[tip].y < hand_landmarks.landmark[pip].y:
        finger_count += 1

return finger_count

```

```

def detect_hand_and_fingers(self, right_hand_landmarks, left_hand_landmarks):
    """
    Функция 1: Определяет какая рука и сколько на ней пальцев

    Returns:
        dict: {"hand": "left_hand"/"right_hand", "fingers": 0-5} или None если руки не
обнаружены
    """
    hand_info = {}

    # Проверяем правую руку
    if right_hand_landmarks:
        right_fingers = self.count_fingers(right_hand_landmarks)
        hand_info["right_hand"] = right_fingers

    # Проверяем левую руку
    if left_hand_landmarks:
        left_fingers = self.count_fingers(left_hand_landmarks)
        hand_info["left_hand"] = left_fingers

    # Если не обнаружено ни одной руки
    if not hand_info:
        return None

    # Возвращаем информацию о всех обнаруженных руках
    return hand_info

```

```

def get_hand_position(self, hand_landmarks):
    """Получение средней позиции ладони"""
    if hand_landmarks is None:
        return None

    # Используем запястье (landmark 0) и среднюю точку ладони
    wrist = hand_landmarks.landmark[0]
    middle_mcp = hand_landmarks.landmark[9] # Основание среднего пальца

    # Средняя позиция
    avg_x = (wrist.x + middle_mcp.x) / 2
    avg_y = (wrist.y + middle_mcp.y) / 2

    return (avg_x, avg_y)

```

```

def detect_movement(self, positions, active_hand):
    """
    Функция 2: Определяет направление движения руки

    Returns:
        str: "UP", "DOWN", "LEFT", "RIGHT" или None если движения недостаточно
    """
    if len(positions) < 2:
        return None

    # Берем несколько последних позиций для более точного определения направления
    if len(positions) >= 5:
        old_positions = list(positions)[:5] # Первые 5 позиций
        new_positions = list(positions)[-5:] # Последние 5 позиций
    else:
        old_positions = [positions[0]]
        new_positions = [positions[-1]]

    # Вычисляем среднее изменение по X и Y
    delta_x_sum = 0
    delta_y_sum = 0

```



```

for old_pos, new_pos in zip(old_positions, new_positions):
    delta_x_sum += (new_pos[0] - old_pos[0])
    delta_y_sum += (new_pos[1] - old_pos[1])

# Среднее изменение
delta_x = delta_x_sum / len(old_positions)
delta_y = delta_y_sum / len(old_positions)

```

8.2. Автомат определения направления движения (A1)

8.2.1. Словесное описание

Каждый кадр добавляется позиция (avg_x, avg_y) в hand_positions.

При g414 вызывается detect_movement.

Сравниваются средние delta_x, delta_y с порогом MOVEMENT_THRESHOLD.

Горизонтальное направление инвертируется для левой/правой руки.

8.2.2. Схема связей и граф переходов

8.2. Автомат A1 — направление движения

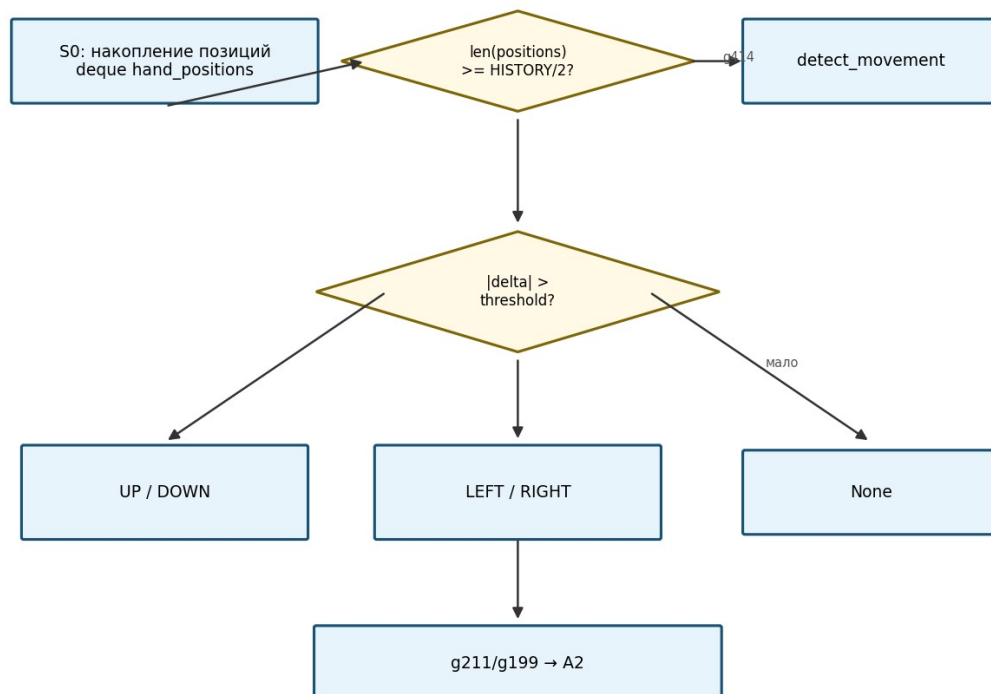


Рисунок 6 — Автомат A1 (направление движения)

8.2.3. Текст функции, реализующей автомат

Фрагмент `detect_movement`, `get_hand_position`, `get_action_from_config`:

```
delta_y = delta_y_sum / len(old_positions)

# Преобразуем в абсолютное значение (пиксели)
delta_x_pixels = delta_x * 1000
delta_y_pixels = delta_y * 1000

# Определяем направление движения
movement_threshold = self.MOVEMENT_THRESHOLD

# Увеличиваем порог для лучшего распознавания LEFT/RIGHT
horizontal_threshold = movement_threshold * 0.8
vertical_threshold = movement_threshold

# Проверяем горизонтальное движение
if abs(delta_x_pixels) > abs(delta_y_pixels):
    # Горизонтальное движение
    if abs(delta_x_pixels) > horizontal_threshold:

        # Добавленное условие для определения направления движения в зависимости от
активной руки
        if active_hand == "left_hand":
            if delta_x_pixels > 0:
                return "RIGHT"
            else:
                return "LEFT"
        else: # right_hand
            if delta_x_pixels < 0:
                return "RIGHT"
            else:
                return "LEFT"
    else:
        # Вертикальное движение
        if abs(delta_y_pixels) > vertical_threshold:
            if delta_y_pixels > 0:
                return "DOWN"
            else:
                return "UP"

return None
```

```

def get_action_from_config(self, hand, fingers, direction):
    """
    Функция 3: Ищет совпадение в конфигурации жестов и возвращает действие

    Args:
        hand: "left_hand" или "right_hand"
        fingers: количество пальцев (1-5)
        direction: "UP", "DOWN", "LEFT", "RIGHT"

    Returns:
        str: Действие из конфига или None если совпадение не найдено
    """

    """Ищет запись по name, age, email и возвращает quantity и key"""

    if self.config is None:
        return None

    for item in self.config:
        if (item.get('hand') == hand and
            item.get('fingers') == fingers and
            item.get('direction') == direction):
            return item.get('action', [])

    return None

```

```

def perform_action(self, action_array):
    """Выполнение системного действия"""

```

8.3. Автомат выполнения действий (A2)

8.3.1. Словесное описание

На входе: active_hand, fingers_count, direction.

get_action_from_config — линейный поиск в JSON.

При найденном action → perform_action (hotkey 1–5 клавиш).

Очистка hand_positions, gesture_active=False.

8.3.2. Схема связей и граф переходов

8.3. Автомат A2 — выполнение действий

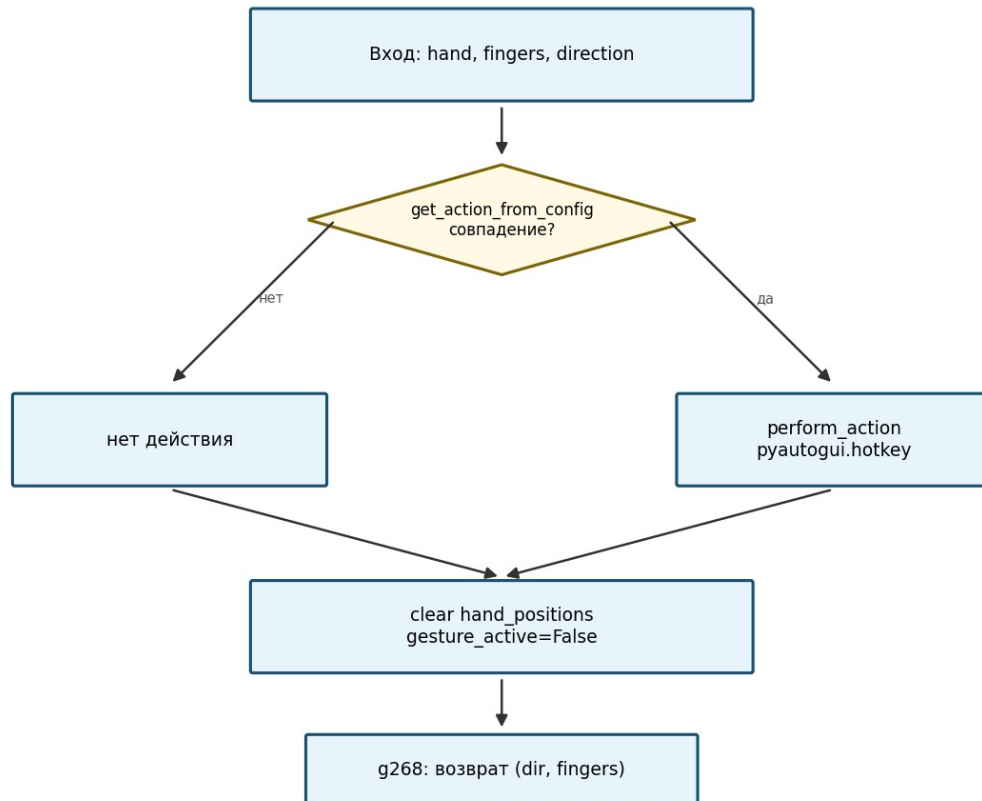


Рисунок 7 — Автомат A2 (выполнение действий)

8.3.3. Текст функции, реализующей автомат

Фрагмент perform_action, process_hands:

```
def perform_action(self, action_array):
    """Выполнение системного действия"""

    if len(action_array) == 1:
        return pyautogui.hotkey(action_array[0])
    elif len(action_array) == 2:
        return pyautogui.hotkey(action_array[0], action_array[1])
    elif len(action_array) == 3:
        return pyautogui.hotkey(action_array[0], action_array[1], action_array[2])
```

```

        elif len(action_array) == 4:
            return
        pyautogui.hotkey(action_array[0],action_array[1],action_array[2],action_array[3])
        elif len(action_array) == 5:
            return
        pyautogui.hotkey(action_array[0],action_array[1],action_array[2],action_array[3],action_array[4])
    ])

```

```

def process_hands(self, right_hand_landmarks, left_hand_landmarks, image=None):
    """
    Основная функция обработки жестов рук

    Args:
        right_hand_landmarks: Landmarks правой руки
        left_hand_landmarks: Landmarks левой руки
        image: Изображение для отрисовки (опционально)

    Returns:
        tuple: (направление движения, количество пальцев) или None
    """
    # 1. Определяем какая рука и сколько пальцев
    hand_info = self.detect_hand_and_fingers(right_hand_landmarks, left_hand_landmarks)

    if not hand_info:
        self.gesture_active = False
        return None

    # Выбираем активную руку для отслеживания (приоритет: правая)
    active_hand = None
    active_hand_landmarks = None
    fingers_count = 0

    if "right_hand" in hand_info and hand_info["right_hand"] > 0:
        active_hand = "right_hand"
        active_hand_landmarks = right_hand_landmarks
        fingers_count = hand_info["right_hand"]
    elif "left_hand" in hand_info and hand_info["left_hand"] > 0:
        active_hand = "left_hand"
        active_hand_landmarks = left_hand_landmarks
        fingers_count = hand_info["left_hand"]
    else:
        self.gesture_active = False
        return None

    # Отображаем информацию о руках на изображении
    if image is not None:

```

```

y_offset = 120
for hand_type, count in hand_info.items():
    color = (0, 255, 0) if hand_type == active_hand else (200, 200, 200)
    hand_label = "r " if hand_type == "right_hand" else "l"
    cv2.putText(image, f"{hand_label}: {count} f",
                (10, y_offset), cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)
    y_offset += 30

# 2. Получаем позицию активной руки и обновляем историю
current_position = self.get_hand_position(active_hand_landmarks)
if current_position:
    self.hand_positions.append(current_position)
    self.gesture_active = True
    self.current_hand = active_hand
    self.current_fingers_count = fingers_count

# 3. Определяем направление движения
if len(self.hand_positions) >= self.HISTORY_LENGTH // 2:
    direction = self.detect_movement(self.hand_positions, active_hand)

    if direction:
        # 4. Ищем действие в конфигурации
        action = self.get_action_from_config(active_hand, fingers_count, direction)

        if action and len(action) > 0:
            # Выполняем действие
            self.perform_action(action)

            # Отображаем информацию о выполненном жесте
            if image is not None:
                direction_text = {
                    "UP": "UP",
                    "DOWN": "DOWN",
                    "LEFT": "LEFT",
                    "RIGHT": "RIGHT"
                }.get(direction, direction)

                #gesture_text = f"gesture: {direction_text}, {fingers_count}
пальцев"

                #cv2.putText(image, gesture_text,
                #            (10, y_offset), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255,
255, 0), 2)

            # Очищаем историю после выполнения действия
            self.hand_positions.clear()
            self.gesture_active = False

        return direction, fingers_count
    else:
        # Сбрасываем историю, если жест не активен
        if len(self.hand_positions) > 0:
            self.hand_positions.clear()
            self.gesture_active = False

return None if not self.gesture_active else (None, fingers_count)

```

```
def is_gesture_active(self):
    """Проверка, активен ли жест в данный момент"""
    return self.gesture_active

def get_current_hand(self):
    """Получение текущей активной руки"""
    return self.current_hand

def get_current_fingers_count(self):
    """Получение текущего количества пальцев"""
    return self.current_fingers_count
```

8.4. Сводная блок-схема process_hands

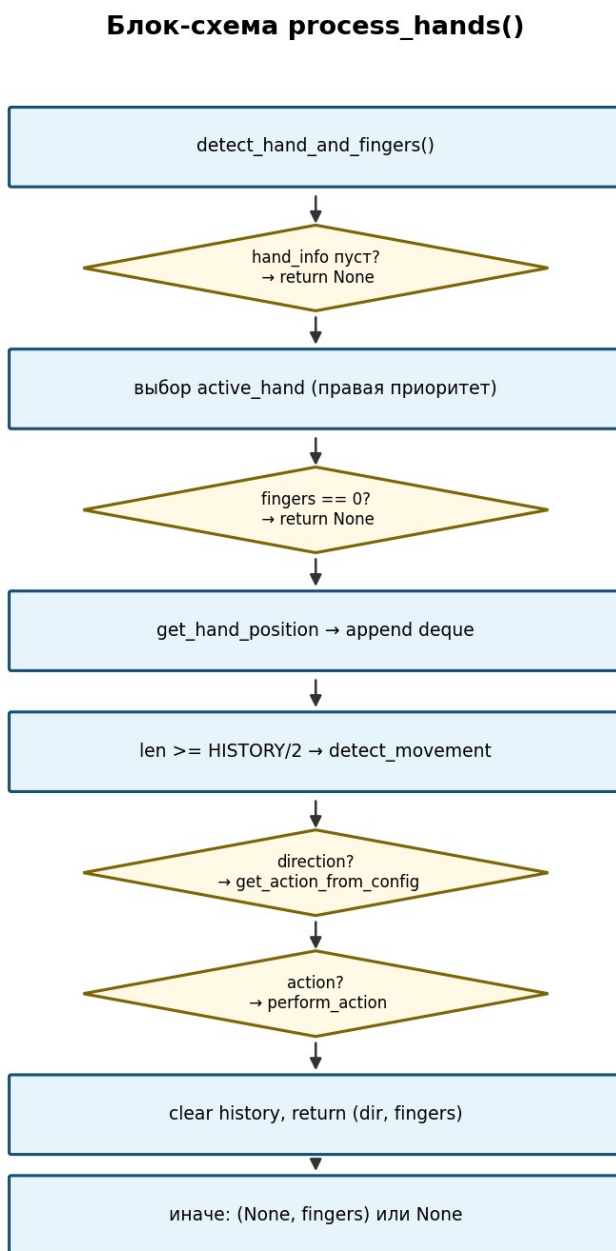


Рисунок 8 — Блок-схема функции process_hands

9. ИСХОДНЫЕ ТЕКСТЫ СИСТЕМОЗАВИСИМОЙ ЧАСТИ

9.1. Модуль распознавания жестов (файл gestures.py)

Листинг gestures.py:

```
import pyautogui
import time
from collections import deque
import cv2
import json
import os

class GestureController:

    def __init__(self, movement_threshold=80, history_length=20, action_cooldown=5,
config_file='gestures_config.json'):
        """
        Инициализация контроллера жестов

        Args:
            config_file: Путь к JSON файлу с конфигурацией жестов
            movement_threshold: Порог движения в пикселях
            history_length: Длина истории позиций для отслеживания
            action_cooldown: Задержка между действиями в секундах
        """
        self.MOVEMENT_THRESHOLD = movement_threshold
        self.ACTION_COOLDOWN = action_cooldown
        self.HISTORY_LENGTH = history_length

        # Для отслеживания позиции руки
        self.hand_positions = deque(maxlen=self.HISTORY_LENGTH)
        self.gesture_active = False
        self.last_action_time = 0
        self.current_hand = None
        self.current_fingers_count = 0

        self.config_file = config_file

        # Загрузка конфигурации
        self.config = None
        if os.path.exists(config_file):
            with open(config_file, 'r', encoding='utf-8') as f:
                self.config = json.load(f)
        # Словарь для быстрого поиска действий
```

```

def count_fingers(self, hand_landmarks):
    """Подсчет поднятых пальцев на руке"""
    if hand_landmarks is None:
        return 0

    # Индексы ключевых точек для пальцев
    finger_tips = [4, 8, 12, 16, 20] # Кончики пальцев
    finger_pips = [3, 6, 10, 14, 18] # Суставы

    finger_count = 0

    # Определяем активную руку (левую или правую)
    # Для этого используем позицию запястья относительно среднего пальца
    # Если запястье находится левее среднего пальца, то это правая рука, иначе левая
    wrist_x = hand_landmarks.landmark[0].x
    middle_mcp_x = hand_landmarks.landmark[9].x

    is_right_hand = wrist_x < middle_mcp_x

    # Большой палец - сравниваем по оси X в зависимости от руки
    if is_right_hand:
        # Для правой руки
        if hand_landmarks.landmark[4].x > hand_landmarks.landmark[3].x:
            finger_count += 1
    else:
        # Для левой руки
        if hand_landmarks.landmark[4].x < hand_landmarks.landmark[3].x:
            finger_count += 1

    # Остальные пальцы - сравниваем по оси Y (кончик выше сустава = палец поднят)
    for tip, pip in zip(finger_tips[1:], finger_pips[1:]):
        if hand_landmarks.landmark[tip].y < hand_landmarks.landmark[pip].y:
            finger_count += 1

    return finger_count


def detect_hand_and_fingers(self, right_hand_landmarks, left_hand_landmarks):
    """
    Функция 1: Определяет какая рука и сколько на ней пальцев

    Returns:
        dict: {"hand": "left_hand"/"right_hand", "fingers": 0-5} или None если руки не
    обнаружены
    """
    hand_info = {}

```

```

# Проверяем правую руку
if right_hand_landmarks:
    right_fingers = self.count_fingers(right_hand_landmarks)
    hand_info["right_hand"] = right_fingers

# Проверяем левую руку
if left_hand_landmarks:
    left_fingers = self.count_fingers(left_hand_landmarks)
    hand_info["left_hand"] = left_fingers

# Если не обнаружено ни одной руки
if not hand_info:
    return None

# Возвращаем информацию о всех обнаруженных руках
return hand_info

```

```

def get_hand_position(self, hand_landmarks):
    """Получение средней позиции ладони"""
    if hand_landmarks is None:
        return None

    # Используем запястье (landmark 0) и среднюю точку ладони
    wrist = hand_landmarks.landmark[0]
    middle_mcp = hand_landmarks.landmark[9] # Основание среднего пальца

    # Средняя позиция
    avg_x = (wrist.x + middle_mcp.x) / 2
    avg_y = (wrist.y + middle_mcp.y) / 2

    return (avg_x, avg_y)

```

```
def detect_movement(self, positions, active_hand):
    """
    Функция 2: Определяет направление движения руки

    Returns:
        str: "UP", "DOWN", "LEFT", "RIGHT" или None если движения недостаточно
    """
    if len(positions) < 2:
        return None

    # Берем несколько последних позиций для более точного определения направления
    if len(positions) >= 5:
        old_positions = list(positions[:5]) # Первые 5 позиций
        new_positions = list(positions[-5:]) # Последние 5 позиций
    else:
        old_positions = [positions[0]]
        new_positions = [positions[-1]]

    # Вычисляем среднее изменение по X и Y
    delta_x_sum = 0
    delta_y_sum = 0

    for old_pos, new_pos in zip(old_positions, new_positions):
        delta_x_sum += (new_pos[0] - old_pos[0])
        delta_y_sum += (new_pos[1] - old_pos[1])

    # Среднее изменение
    delta_x = delta_x_sum / len(old_positions)
    delta_y = delta_y_sum / len(old_positions)

    # Преобразуем в абсолютное значение (пиксели)
    delta_x_pixels = delta_x * 1000
    delta_y_pixels = delta_y * 1000

    # Определяем направление движения
    movement_threshold = self.MOVEMENT_THRESHOLD

    # Увеличиваем порог для лучшего распознавания LEFT/RIGHT
    horizontal_threshold = movement_threshold * 0.8
    vertical_threshold = movement_threshold

    # Проверяем горизонтальное движение
    if abs(delta_x_pixels) > abs(delta_y_pixels):
        # Горизонтальное движение
        if abs(delta_x_pixels) > horizontal_threshold:

            # Добавленное условие для определения направления движения в зависимости от
            # активной руки
            if active_hand == "left_hand":
                if delta_x_pixels > 0:
                    return "RIGHT"
                else:
                    return "LEFT"
            else: # right_hand
                if delta_x_pixels < 0:
                    return "RIGHT"
                else:
                    return "LEFT"
        else:
            return None
    else:
        # Вертикальное движение
        if abs(delta_y_pixels) > vertical_threshold:
            if delta_y_pixels > 0:
                return "UP"
            else:
                return "DOWN"
        else:
            return None
```

```

        return "LEFT"
    else:
        # Вертикальное движение
        if abs(delta_y_pixels) > vertical_threshold:
            if delta_y_pixels > 0:
                return "DOWN"
            else:
                return "UP"

    return None

```

```

def get_action_from_config(self, hand, fingers, direction):
    """
    Функция 3: Ищет совпадение в конфигурации жестов и возвращает действие

    Args:
        hand: "left_hand" или "right_hand"
        fingers: количество пальцев (1-5)
        direction: "UP", "DOWN", "LEFT", "RIGHT"

    Returns:
        str: Действие из конфига или None если совпадение не найдено
    """

    """Ищет запись по name, age, email и возвращает quantity и key"""

    if self.config is None:
        return None

    for item in self.config:
        if (item.get('hand') == hand and
            item.get('fingers') == fingers and
            item.get('direction') == direction):
            return item.get('action', [])

    return None

```

```

def perform_action(self, action_array):
    """Выполнение системного действия"""

    if len(action_array) == 1:
        return pyautogui.hotkey(action_array[0])
    elif len(action_array) == 2:
        return pyautogui.hotkey(action_array[0], action_array[1])
    elif len(action_array) == 3:
        return pyautogui.hotkey(action_array[0], action_array[1], action_array[2])
    elif len(action_array) == 4:
        return
    pyautogui.hotkey(action_array[0], action_array[1], action_array[2], action_array[3])
    elif len(action_array) == 5:
        return
    pyautogui.hotkey(action_array[0], action_array[1], action_array[2], action_array[3], action_array[4])
]

```

```

def process_hands(self, right_hand_landmarks, left_hand_landmarks, image=None):
    """
    Основная функция обработки жестов рук

    Args:
        right_hand_landmarks: Landmarks правой руки
        left_hand_landmarks: Landmarks левой руки
        image: Изображение для отрисовки (опционально)

    Returns:
        tuple: (направление движения, количество пальцев) или None
    """
    # 1. Определяем какая рука и сколько пальцев
    hand_info = self.detect_hand_and_fingers(right_hand_landmarks, left_hand_landmarks)

    if not hand_info:
        self.gesture_active = False

```

```

        return None

# Выбираем активную руку для отслеживания (приоритет: правая)
active_hand = None
active_hand_landmarks = None
fingers_count = 0

if "right_hand" in hand_info and hand_info["right_hand"] > 0:
    active_hand = "right_hand"
    active_hand_landmarks = right_hand_landmarks
    fingers_count = hand_info["right_hand"]
elif "left_hand" in hand_info and hand_info["left_hand"] > 0:
    active_hand = "left_hand"
    active_hand_landmarks = left_hand_landmarks
    fingers_count = hand_info["left_hand"]
else:
    self.gesture_active = False
    return None

# Отображаем информацию о руках на изображении
if image is not None:
    y_offset = 120
    for hand_type, count in hand_info.items():
        color = (0, 255, 0) if hand_type == active_hand else (200, 200, 200)
        hand_label = "r " if hand_type == "right_hand" else "l"
        cv2.putText(image, f"{hand_label}: {count} f",
                    (10, y_offset), cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)
        y_offset += 30

# 2. Получаем позицию активной руки и обновляем историю
current_position = self.get_hand_position(active_hand_landmarks)
if current_position:
    self.hand_positions.append(current_position)
    self.gesture_active = True
    self.current_hand = active_hand
    self.current_fingers_count = fingers_count

# 3. Определяем направление движения
if len(self.hand_positions) >= self.HISTORY_LENGTH // 2:
    direction = self.detect_movement(self.hand_positions, active_hand)

    if direction:
        # 4. Ищем действие в конфигурации
        action = self.get_action_from_config(active_hand, fingers_count, direction)

        if action and len(action) > 0:
            # Выполняем действие
            self.perform_action(action)

# Отображаем информацию о выполненном жесте
if image is not None:
    direction_text = {
        "UP": "UP",
        "DOWN": "DOWN",
        "LEFT": "LEFT",
        "RIGHT": "RIGHT"
    }.get(direction, direction)

```

```

        #gesture_text = f"gesture: {direction_text}, {fingers_count}"
пальцев"
        #cv2.putText(image, gesture_text,
        #              (10, y_offset), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255,
255, 0), 2)

```

```

        # Очищаем историю после выполнения действия
        self.hand_positions.clear()
        self.gesture_active = False

```

```

        return direction, fingers_count

```

```

    else:

```

```

        # Сбрасываем историю, если жест не активен
        if len(self.hand_positions) > 0:
            self.hand_positions.clear()
        self.gesture_active = False

```

```

    return None if not self.gesture_active else (None, fingers_count)

```

```

def is_gesture_active(self):
    """Проверка, активен ли жест в данный момент"""
    return self.gesture_active

```

```

def get_current_hand(self):
    """Получение текущей активной руки"""
    return self.current_hand

```

```

def get_current_fingers_count(self):
    """Получение текущего количества пальцев"""
    return self.current_fingers_count

```

```

def create_default_gesture_controller():
    """Создание контроллера жестов с настройками по умолчанию"""
    return GestureController(
        config_file="gestures_config.json",
        movement_threshold=30,
        history_length=15, # Увеличено для более плавного отслеживания
        action_cooldown=1
    )

```


9.2. Консольный клиент (файл main.py)

Листинг main.py:

```
import cv2

import mediapipe as mp

from gestures import GestureController

# Grabbing the Holistic Model from Mediapipe and Initializing the Model
mp_holistic = mp.solutions.holistic
holistic_model = mp_holistic.Holistic(
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)

# Initializing the drawing utils
mp_drawing = mp.solutions.drawing_utils

# (0) in VideoCapture is used to connect to your computer's default camera
capture = cv2.VideoCapture(0)

# Initializing current time and previous time for calculating the FPS
previousTime = 0
currentTime = 0

# Initialize gesture controller
gesture_controller = GestureController()

while capture.isOpened():
    # Capture frame by frame
    ret, frame = capture.read()
    if not ret:
        break

    # Converting from BGR to RGB
    image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # Making predictions using holistic model
    image.flags.writeable = False
    results = holistic_model.process(image)
    image.flags.writeable = True

    # Converting back the RGB image to BGR
    image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)

    # Process gestures using the gesture controller
    gesture_info = gesture_controller.process_hands(
        results.right_hand_landmarks,
        results.left_hand_landmarks,
        image
    )
```

```

# Display gesture information if available

# Status text

# Drawing the Facial Landmarks

# Drawing Right hand Land Marks
if results.right_hand_landmarks:
    mp_drawing.draw_landmarks(
        image,
        results.right_hand_landmarks,
        mp_holistic.HAND_CONNECTIONS,
        mp_drawing.DrawingSpec(color=(255,0,0), thickness=2, circle_radius=2),
        mp_drawing.DrawingSpec(color=(0,255,0), thickness=2, circle_radius=2)
    )

# Drawing Left hand Land Marks
if results.left_hand_landmarks:
    mp_drawing.draw_landmarks(
        image,
        results.left_hand_landmarks,
        mp_holistic.HAND_CONNECTIONS,
        mp_drawing.DrawingSpec(color=(0,0,255), thickness=2, circle_radius=2),
        mp_drawing.DrawingSpec(color=(0,255,0), thickness=2, circle_radius=2)
    )

# Display the resulting image
image = cv2.flip(image, 1)

if gesture_info:
    direction, fingers_count = gesture_info
    if direction:
        cv2.putText(image, f"Gesture: {direction}",
            (10, 200), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0), 2)
        cv2.putText(image, f"Fingers: {fingers_count}",
            (10, 240), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

    status_text = "Gesture: ACTIVE" if gesture_controller.is_gesture_active() else "Gesture:
INACTIVE"
    status_color = (0, 255, 0) if gesture_controller.is_gesture_active() else (0, 0, 255)
    cv2.putText(image, status_text, (10, 280),
        cv2.FONT_HERSHEY_SIMPLEX, 1, status_color, 2)

cv2.imshow("Gesture Control ", image)

# Enter key 'q' to break the loop
if cv2.waitKey(5) & 0xFF == ord('q'):
    break

```

```
# When all the process is done
# Release the capture and destroy all windows
capture.release()
cv2.destroyAllWindows()
```

9.3. Графический клиент (файл gui_app.py)

Полный текст gui_app.py приведён в репозитории; ниже — заголовок и класс VideoThread.

Фрагмент gui_app.py:

```
import sys
import json
import os
import cv2
import numpy as np
from PyQt6.QtWidgets import (
    QApplication, QMainWindow, QPushButton, QVBoxLayout, QWidget, QLabel,
    QDialog, QListWidget, QListWidgetItem, QHBoxLayout, QMessageBox,
    QSpinBox, QComboBox, QLineEdit, QRadioButton, QButtonGroup, QGroupBox,
    QFormLayout, QDialogButtonBox
)
from PyQt6.QtCore import Qt, QThread, pyqtSignal, QTimer
from PyQt6.QtGui import QImage, QPixmap
from gestures import GestureController

class VideoThread(QThread):
    change_pixmap_signal = pyqtSignal(np.ndarray)
    gesture_info_signal = pyqtSignal(dict)

    def __init__(self, gesture_controller):
        super().__init__()
        self.gesture_controller = gesture_controller
        self._run_flag = True
        self.capture = cv2.VideoCapture(0)

    def run(self):
        import mediapipe as mp
        mp_holistic = mp.solutions.holistic
        holistic_model = mp_holistic.Holistic(
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5
        )
        mp_drawing = mp.solutions.drawing_utils

        while self._run_flag:
            ret, frame = self.capture.read()
            if not ret:
                continue

            image_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            image_rgb.flags.writeable = False
            results = holistic_model.process(image_rgb)
            image_rgb.flags.writeable = True
            image_bgr = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2BGR)

            gesture_info = self.gesture_controller.process_hands(
                results.right_hand_landmarks,
```

```

        results.left_hand_landmarks,
        image_bgr
    )

    if results.right_hand_landmarks:
        mp_drawing.draw_landmarks(
            image_bgr,
            results.right_hand_landmarks,
            mp_holistic.HAND_CONNECTIONS,
            mp_drawing.DrawingSpec(color=(255,0,0), thickness=2, circle_radius=2),
            mp_drawing.DrawingSpec(color=(0,255,0), thickness=2, circle_radius=2)
        )
    if results.left_hand_landmarks:
        mp_drawing.draw_landmarks(
            image_bgr,
            results.left_hand_landmarks,
            mp_holistic.HAND_CONNECTIONS,
            mp_drawing.DrawingSpec(color=(0,0,255), thickness=2, circle_radius=2),
            mp_drawing.DrawingSpec(color=(0,255,0), thickness=2, circle_radius=2)
        )

    # Переворачиваем изображение для естественного отображения
    image_bgr = cv2.flip(image_bgr, 1)

    # Формируем информацию о жесте
    if gesture_info:
        direction, fingers_count = gesture_info
        if direction:
            self.gesture_info_signal.emit({
                'direction': direction,
                'fingers': fingers_count,
                'gesture_active': self.gesture_controller.is_gesture_active()
            })
        else:
            self.gesture_info_signal.emit({
                'direction': None,
                'fingers': fingers_count,
                'gesture_active': self.gesture_controller.is_gesture_active()
            })
    else:
        self.gesture_info_signal.emit({
            'direction': None,
            'fingers': 0,
            'gesture_active': False
        })

    # Отправляем кадр
    self.change_pixmap_signal.emit(image_bgr)

    self.capture.release()
    holistic_model.close()
    cv2.destroyAllWindows()

    def stop(self):
        self._run_flag = False
        self.wait()

# ----- Оконное отслеживание жестов (камера) -----
class CameraWindow(QMainWindow):

```

```

def __init__(self, gesture_controller):
    super().__init__()
    self.gesture_controller = gesture_controller
    self.setWindowTitle("Gesture Control - Camera")
    self.setGeometry(100, 100, 800, 600)

    central_widget = QWidget()
    self.setCentralWidget(central_widget)
    layout = QVBoxLayout(central_widget)

    # Виджет для видео
    self.image_label = QLabel()
    self.image_label.setAlignment(Qt.AlignmentFlag.AlignCenter)

```

9.4. Конфигурация жестов (файл gestures_config.json)

Листинг gestures_config.json:

```

[
  {
    "hand": "left_hand",
    "fingers": 3,
    "direction": "DOWN",
    "action": [
      "win",
      "d"
    ]
  },
  {
    "hand": "left_hand",
    "fingers": 3,
    "direction": "UP",
    "action": [
      "win",
      "d"
    ]
  },
  {
    "hand": "left_hand",
    "fingers": 2,
    "direction": "RIGHT",
    "action": [
      "alt",
      "shift",
      "tab"
    ]
  },
  {
    "hand": "left_hand",
    "fingers": 2,
    "direction": "LEFT",
    "action": [
      "alt",
      "tab"
    ]
  },
  {
    "hand": "right_hand",
    "fingers": 5,
    "direction": "DOWN",
    "action": [
      "volumedown"
    ]
  }
]

```

```

    ]
  },
  {
    "hand": "right_hand",
    "fingers": 5,
    "direction": "UP",
    "action": [
      "volumeup"
    ]
  },
  {
    "hand": "right_hand",
    "fingers": 5,
    "direction": "RIGHT",
    "action": [
      "volumeup"
    ]
  },
  {
    "hand": "right_hand",
    "fingers": 4,
    "direction": "RIGHT",
    "action": [
      "nexttrack"
    ]
  },
  {
    "hand": "right_hand",
    "fingers": 4,
    "direction": "LEFT",
    "action": [
      "prevtrack"
    ]
  },
  {
    "hand": "right_hand",
    "fingers": 4,
    "direction": "UP",
    "action": [
      "playpause"
    ]
  },
  {
    "hand": "right_hand",
    "fingers": 4,
    "direction": "DOWN",
    "action": [
      "pause"
    ]
  }
]

```

10. ПРОТОКОЛЫ ФУНКЦИОНИРОВАНИЯ ПОДСИСТЕМЫ

10.1. Протоколы функционирования модуля распознавания

10.1.1. Диагностирующий (полный) протокол

№	Операция	Ожидаемый результат	Статус
1	pip install -r requirements.txt	Установка без ошибок	
2	python main.py	Окно камеры, скелет руки	
3	Правая рука, 5 пальцев	Подпись r: 5 f, ACTIVE	
4	Движение вверх (5 пальцев)	volumeup	
5	Движение вниз	volumedown	
6	4 пальца, влево/вправо	prevtrack / nexttrack	
7	Левая рука, 2 пальца, влево	alt+tab	
8	Убрать руки	INACTIVE	
9	Клавиша q	Закрытие окна	
10	python gui_app.py	Видео в Qt-окне	
11	Edit config → Save	Запись в JSON	
12	Повтор жеста	Новое действие	

10.1.2. Проверяющий (короткий) протокол

1. Запуск main.py, рука в кадре — есть разметка.
2. Один жест из таблицы 2.4 — срабатывает системное действие.
3. Выход q — процесс завершается.

11. ТРЕБОВАНИЯ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ

Функциональные: распознавание 1–2 рук, 4 направления, настраиваемые действия, два клиента.

Надёжность: отсутствие JSON не приводит к падению; сброс истории после жеста.

Состав ТС: ПК Windows 10/11, веб-камера USB, разрешение экрана $\geq 1280 \times 720$.

Совместимость: Python 3.12+, MediaPipe 0.10.21, JSON UTF-8, PyAutoGUI.

12. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

1. Настоящий документ DOCUMENTATION.docx.
2. Комплект блок-схем PNG в doc/diagrams/.
3. Скрипт сборки doc/build_docx.py.
4. Актуальные requirements.txt и README.md.

13. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

Показатель	Оценка
Трудоёмкость сопровождения	Низкая
Стоимость лицензий	0 (открытые библиотеки)
Аппаратные требования	Стандартный ПК + камера

14. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

Этап	Содержание	Результат
1	ТЗ, выбор MediaPipe	Спецификация жестов
2	gestures.py, A0–A2	Ядро
3	main.py	Консольный прототип
4	gestures_config.json	Настраиваемые действия
5	gui_app.py	GUI и редактор
6	Документация, блок-схемы	Папка doc/

15. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ

1. Проверка комплектности файлов проекта.
2. Прохождение проверяющего протокола (п. 10.1.2).
3. Сверка блок-схем с исходным кодом.
4. Проверка соответствия JSON и фактических жестов.

16. УСТАНОВКА И ЭКСПЛУАТАЦИЯ

Команды установки и запуска:

```
cd gesture-control
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pip install opencv-python PyQt6
python gui_app.py
```

PyAutoGUI на Windows требует доступа к фокусу окна. Зеркальное отображение `cv2.flip(image, 1)` согласовано с логикой `detect_movement`.

ПРИЛОЖЕНИЕ А. Допустимые имена клавиш PyAutoGUI (фрагмент)

win, d, alt, shift, tab, volumeup, volumedown, nexttrack, prevtrack, playpause, pause, ctrl, enter, esc, space, f1–f12, стрелки up/down/left/right — полный список см. PyAutoGUI.

ПРИЛОЖЕНИЕ Б. Индексы landmarks руки MediaPipe

Индекс	Анатомия
0	Запястье
4	Кончик большого
8, 12, 16, 20	Кончики пальцев
3, 6, 10, 14, 18	PIP суставы
9	MCP среднего пальца

Каталог блок-схем

Файл	Содержание
01_structural_overview.png	§ 3.1 Общая структура
02_module_interaction.png	§ 3.2 Взаимодействие за кадр
03_main_loop.png	Цикл main.py
04_automaton_A0.png	Автомат выбора руки
05_automaton_A1.png	Автомат движения
06_automaton_A2.png	Автомат действий
07_process_hands.png	process_hands()
08_gui_architecture.png	Структура GUI